



Architectural Decisions



Architectural Decision

Context:

In addition to the nutritional data retrieved from the external API, we need to store the data generated by our own users: profiles, goals, and most importantly, the history of all logged meals. This data possesses a clear structure and inherent relationships (a user has multiple meals; a meal consists of multiple food items). The choice of database type (Relational/SQL vs. Non-Relational/NoSQL) is crucial and will directly impact data consistency, performance, and the complexity of the queries we can execute.



Architectural Decision

Decision: We will use a Relational (SQL) database, specifically PostgreSQL.

Architectural Decision



❑ Relational Database - PostgreSQL:

- **What is it?** It is a type of database that stores information in tables with rows and columns, using a predefined relational schema (the table structure is fixed). It excels at managing "relationships" between tables, for example, connecting the users table to the meals table through a user ID.

❑ Arguments in Favor:

- **Complex and Powerful Queries:** The SQL language is extremely mature and powerful for asking complex questions of the data.
- **Predictable Data Structure:** The data is naturally structured (a user has an email, a meal has a date, etc.). A rigid schema enforces data consistency.

❑ Arguments Against:

- **Lower Schema Flexibility:** If we want to add a new information field to all previously saved meals, the process (called a migration) is more formal and can be more complex than in some NoSQL databases.